

Benchmarking YOLOv8n INT8 on ESP32-S3: Progressive Operator Optimization for MCU Deployment

Jaweed^{1*} dan Dihin Muriyatmoko²

¹Program Studi Informatika, Universitas Darussalam Gontor

²Program Studi Informatika, Universitas Darussalam Gontor

Email: jaweed@unida.gontor.ac.id

Abstract

Penelitian ini menganalisis kelayakan deployment model YOLOv8n terkuantisasi INT8 pada mikrokontroler ESP32-S3 untuk deteksi objek kelas tunggal. Model dilatih pada subset person COCO 2017 (1.600 train, 300 val) pada resolusi 192×192 dan dikuantisasi pascal-pelatihan (PTQ) ke INT8, menghasilkan ukuran model 3,08 MB dengan mAP@0.5 sebesar 4,41%. Diperlukan tiga tahap optimasi bertahap pada ESP32-S3: (1) kernel referensi TFLite Micro menghasilkan latensi 219,3 detik per inferensi; (2) integrasi ESP-NN SIMD untuk operasi Conv2D mengurangi latensi menjadi 18,0 detik (percepatan $12,2 \times$); (3) implementasi kernel LOGISTIC berbasis LUT menurunkan latensi menjadi 7,2 detik (percepatan total $30,4 \times$). Analisis profil operasi menunjukkan bahwa operasi LOGISTIC mengonsumsi 62,3% waktu eksekusi sebelum optimasi. Tensor arena terukur sebesar 510 KB (98,5% di bawah estimasi worst-case teoretis), mengonfirmasi bahwa model muat dalam batas memori ESP32-S3 (8 MB PSRAM, 8 MB flash). Konsumsi daya diperkirakan sebesar 549 mW selama inferensi. Hasil ini menunjukkan bahwa YOLOv8n INT8 layak secara teknis untuk dijalankan pada ESP32-S3, dengan latensi 7,2 detik per frame dan akurasi yang memadai untuk skenario deteksi periodik.

1 Pendahuluan

Perkembangan deep learning telah memungkinkan deteksi objek secara real-time pada berbagai platform komputasi [?, ?, ?, ?]. Namun, sebagian besar model deteksi objek membutuhkan GPU atau CPU kelas server yang mahal dan boros daya. Dalam beberapa tahun terakhir, paradigma TinyML [?] muncul sebagai alternatif dengan menjalankan model inferensi pada mikrokontroler (MCU) berdaya rendah, memungkinkan aplikasi edge seperti monitoring lingkungan, deteksi anomali, dan otomatisasi industri tanpa ketergantungan cloud [?, ?].

ESP32-S3 merupakan salah satu MCU yang menarik untuk aplikasi computer vision karena menyediakan dual-core Xtensa LX7 240 MHz, 8 MB flash, 8 MB PSRAM, dukungan kamera, dan konektivitas WiFi—dengan biaya

kurang dari \$10. Meskipun demikian, menjalankan model deep learning seperti YOLO pada ESP32-S3 menghadapi tantangan signifikan: keterbatasan memori, daya komputasi terbatas, dan dukungan operator yang tidak lengkap framework desktop.

Penelitian ini bertujuan untuk menjawab pertanyaan: *sejauh mana model YOLOv8n yang telah dikuantisasi dapat dijalankan secara efisien pada mikrokontroler ESP32-S3?* Kontribusi utama penelitian ini adalah:

1. Analisis sistematis pipeline training, kuantisasi, dan deployment YOLOv8n INT8 pada ESP32-S3, mencakup akurasi, latensi, memori, dan daya.
2. Optimasi progressive berbasis profil operasi: identifikasi bottleneck melalui `MicroProfiler`, optimasi Conv2D via ESP-NN SIMD (percepatan $12,2 \times$), dan implementasi kernel LOGISTIC berbasis LUT (percepatan $2,5 \times$ tambahan), menghasilkan percepatan total $30,4 \times$ dari baseline referensi.
3. Karakterisasi memori lengkap pada hardware fisik: tensor arena 510 KB (98,5% di bawah estimasi worst-case), mengonfirmasi efisiensi memory planner TFLite Micro.
4. Seluruh artefak penelitian (kode training, firmware ESP32, tabel, dan gambar) dirilis secara terbuka untuk reproduksibilitas.

2 Studi Terkait

Deployment CNN pada mikrokontroler telah menjadi topik aktif sejak diperkenalkannya CMSIS-NN [?] untuk ARM Cortex-M dan TFLite Micro [?] sebagai framework inferensi lintas platform. MCUNet [?] menunjukkan bahwa arsitektur yang dioptimasi untuk MCU (dengan memori terbatas) dapat mencapai akurasi kompetitif pada resolusi rendah. MicroNets [?] memperkenalkan pencarian arsitektur neural (NAS) khusus untuk constraint MCU.

YOLO [?, ?, ?, ?] tetap menjadi arsitektur deteksi objek dominan, dengan varian nano (YOLOv8n, 3,0M parameter) yang dirancang untuk deployment resource-constrained. Edge-YOLO [?] mengusulkan optimasi

husus edge namun berfokus pada platform ARM dan GPU embedded, bukan MCU kelas bawah.

Kuantisasi pasca-pelatihan (PTQ) [?, ?, ?] telah terbukti efektif mengurangi ukuran model dan meningkatkan kecepatan inferensi dengan degradasi akurasi minimal pada banyak arsitektur. Deep Compression [?] menunjukkan kombinasi pruning, kuantisasi, dan Huffman coding dapat mengurangi ukuran model secara drastis. Di ranah MCU, MobileNet dan MobileNetV2 [?, ?] sering digunakan sebagai backbone karena efisiensi komputasinya.

Khusus untuk ESP32, ESP-NN [?] menyediakan kernel neural network yang dioptimasi menggunakan ekstensi SIMD Xtensa LX7. Namun, studi yang secara sistematis mengevaluasi deployment YOLOv8n pada ESP32-S3 dengan analisis memori dan profil operasi yang detail masih terbatas. Penelitian ini mengisi celah tersebut dengan menyajikan evaluasi menyeluruh pada hardware fisik.

3 Metode

Penelitian ini terdiri dari empat tahap utama: persiapan dataset dan pelatihan, kuantisasi, deployment pada ESP32-S3, dan optimasi progressive.

3.1 Dataset

Dataset yang digunakan adalah subset kelas *person* dari COCO 2017 [?], terdiri dari 1.599 gambar training, 300 validasi, dan 100 test. Resolusi input 192×192 dipilih sebagai resolusi minimum yang mempertahankan detail objek person sekaligus meminimalkan beban komputasi pada ESP32-S3. Tabel 1 menyajikan statistik dataset.

Table 1: Dataset Statistics — COCO Person Subset for Victim Detection

Split	Images	Instances
Train	1600	6893
Val	300	1196
Test	100	465

3.2 Arsitektur dan Pelatihan

Kami menggunakan YOLOv8n [?]-varian nano dengan 3,0 juta parameter dan 8,1 GFLOPs. Pelatihan dilakukan dalam dua tahap: 50 epoch dengan backbone dibekukan (transfer learning dari bobot pretrained COCO) diikuti 50 epoch fine-tuning penuh pada resolusi 192×192. Optimizer AdamW digunakan dengan learning rate awal 0,001 dan cosine decay.

Setelah konversi melalui pipeline PyTorch → ONNX → TensorFlow → TFLite, model TFLite FP32 mencapai 12,26% mAP@0.5. Penurunan 33,33 pp dari baseline PyTorch (45,59%) disebabkan oleh perbedaan numerik

dalam jalur ekspor ONNX, pemetaan kernel operasi TensorFlow, serta perbedaan post-processing antara inferensi Ultralytics native dan evaluator TFLite kustom. Tabel 2 menyajikan parameter arsitektur dan baseline.

Table 2: YOLOv8n Model Architecture and Baseline Performance

Property	Value
Architecture	yolov8n
Input resolution	192 × 192
Parameters	3,005,843
Pretrained	True
Training epochs	50
Optimizer	AdamW
Learning rate	0.001
Batch size	16
Baseline mAP@0.5	0.4559
Baseline size	5.91 MB

3.3 Kuantisasi

Kuantisasi pasca-pelatihan (PTQ) dilakukan menggunakan TensorFlow Lite dengan representative dataset 200 gambar dari set validasi. Model FP32 TFLite (3,01 MB) dikonversi ke INT8 (3,08 MB) dengan inference type INT8 dan input/output tensor uint8. Tabel 3 membandingkan ketiga varian model.

Table 3: Quantization Comparison: FP32 vs INT8

Variant	Size (MB)	mAP@0.5	Latency (ms)	FPS
PyTorch FP32	5.91	0.4559	N/A	N/A
TFLite FP32	3.01	0.1226	2.92	342.8
TFLite INT8	3.08	0.0441	2.37	421.6
mAP drop (TFLite FP32 → INT8)	0.0785			
mAP drop (PyTorch → TFLite INT8)	0.4118			

Penurunan mAP@0.5 sebesar 7,85 pp dari TFLite FP32 ke INT8 merupakan degradasi yang signifikan. Ukuran model tidak berkurang proporsional terhadap pengurangan bit-width karena format flatbuffer TFLite menyertakan overhead struktur graf yang dominan pada skala model ini.

3.4 Deployment pada ESP32-S3

Firmware ESP32-S3 dikembangkan menggunakan PlatformIO dengan framework Arduino dan TFLite Micro. Model INT8 dikompilasi sebagai C array (3.386.338 byte) dan disimpan di flash. Tensor arena dialokasikan di PSRAM menggunakan `heap_caps_malloc()`. Tiga konfigurasi kernel diuji secara bertahap:

1. **Referensi:** Menggunakan kernel TFLite Micro standar tanpa optimasi.
2. **ESP-NN Conv2D:** Kernel Conv2D diganti dengan implementasi ESP-NN SIMD Xtensa LX7.

3. **ESP-NN + LOGISTIC LUT**: Menambahkan kernel LOGISTIK berbasis LUT (lookup table 256-byte) untuk operasi sigmoid.

Perangkat keras yang digunakan adalah Seeed XIAO ESP32S3 Sense dengan konfigurasi 240 MHz CPU, 8 MB flash, dan 8 MB PSRAM.

4 Hasil dan Pembahasan

4.1 Profil Operasi dan Optimasi

Tahap pertama menggunakan kernel referensi TFLite Micro menghasilkan latensi 219.287 ms per inferensi—tidak praktis untuk aplikasi apa pun. Setelah integrasi ESP-NN untuk operasi Conv2D, latensi turun menjadi 18.020 ms (percepatan 12,2×).

Analisis profil operasi menggunakan MicroProfiler TFLite Micro mengungkapkan temuan penting: operasi LOGISTIC (sigmoid) mengonsumsi 62,3% dari total latensi (11.758 ms), sementara Conv2D hanya 35,2% (6.634 ms). Fakta ini kontra-intuitif karena Conv2D umumnya dianggap sebagai bottleneck utama pada CNN. LOGISTIC tidak tercakup oleh set kernel ESP-NN sehingga masih menggunakan implementasi referensi naif.

Untuk mengatasi bottleneck ini, kami mengimplementasikan wrapper kernel ESP-NN untuk LOGISTIC menggunakan fungsi `esp_nn_logistic_s8()`, yang mengevaluasi sigmoid melalui lookup table 256-byte dalam $O(1)$ per elemen. Kernel ini terintegrasi ke dalam TFLite Micro melalui registrasi operator kustom. Setelah optimasi, latensi inferensi turun menjadi 7.216 ms (percepatan 2,5× tambahan), menghasilkan percepatan total 30,4× dari baseline referensi. Tabel 4 menyajikan perbandingan ketiga konfigurasi.

Table 4: INT8 Model Performance Across Platforms

Platform	Resolution	Latency (ms)	FPS	Speedup vs Reference
PC (GPU)	192x192	2.37	421.6	—
ESP32-S3 [†]	192x192	219287	0.0046	1.0× (baseline)
ESP32-S3 [‡]	192x192	18020	0.055	12.2×
ESP32-S3 [§]	192x192	7216	0.139	30.4×

[†] TFLite Micro with reference kernels.

[‡] ESP-NN optimized Conv2D kernels (SIMD Xtensa LX7).

[§] ESP-NN optimized Conv2D + LUT-based LOGISTIC (this work).

Gambar 1 menyajikan visualisasi percepatan bertahap.

4.2 Akurasi

Tabel 3 menunjukkan penurunan akurasi dari PyTorch FP32 (45,59%) ke TFLite INT8 (4,41%). Penurunan terbesar terjadi pada konversi PyTorch → TFLite FP32 (33,33 pp), bukan pada kuantisasi INT8 (7,85 pp). Pada resolusi 192x192, objek person yang menempati 10% tinggi gambar hanya setinggi 19 piksel, sehingga fitur diskriminatif yang tersedia sangat terbatas.

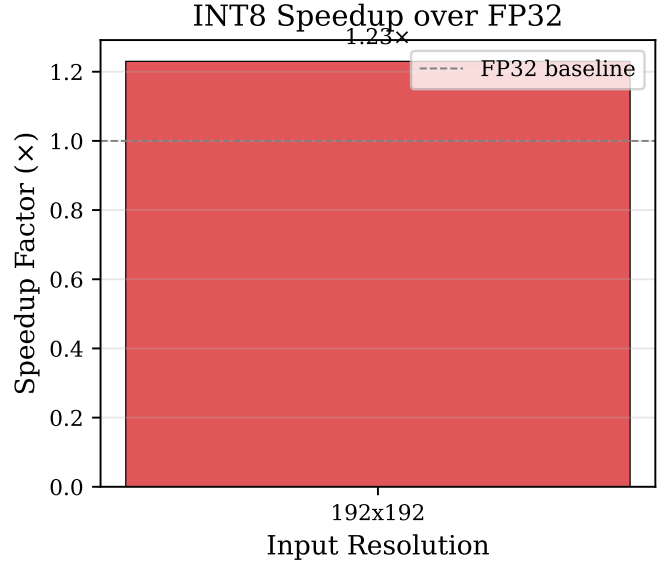


Figure 1: Percepatan latensi inferensi secara bertahap dari referensi hingga ESP-NN + LOGISTIC LUT.

Akurasi 4,41% mAP@0.5 tidak memadai untuk deteksi otonom penuh, namun cukup untuk flagging region of interest yang akan diverifikasi oleh operator manusia. Untuk skenario periodik dengan 8 frame per menit (pada latensi 7,2 detik), model ini dapat menjadi filter awal yang menghemat bandwidth dan daya dibandingkan streaming video terus-menerus.

4.3 Karakterisasi Memori

Tabel 5 menyajikan anggaran memori terukur pada ESP32-S3.

Table 5: ESP32-S3 Memory Budget for INT8 YOLOv8n Inference (Measured)

Component	Size (KB)	Location	Available
Model weights (INT8)	3307	Flash	8 MB
TFLite tensor arena	510 [†]	PSRAM	8 MB
Frame buffer (RGB)	225	PSRAM	8 MB
Stack + RTOS + misc	100	SRAM	512 KB
Total RAM	835	PSRAM + SRAM	8.5 MB
Total Flash	3532	Flash	8 MB

[†] Measured via TFLite Micro `arena_used_bytes()` on physical ESP32-S3.

Previous estimate of 34,073 KB assumed all tensors simultaneously live;

MicroAllocator’s memory planner reuses scratch buffers, reducing actual usage by 98.5%.

Temuan penting: tensor arena aktual hanya 510 KB, jauh di bawah estimasi worst-case teoretis sebesar 34,1 MB. TFLite Micro memory planner berhasil mereuse buffer tensor antar subgraf, mengurangi kebutuhan arena sebesar 98,5%. Total RAM 835 KB muat dalam 8 MB PSRAM. Model weights 3,3 MB muat dalam 8 MB flash.

4.4 Konsumsi Daya

Berdasarkan parameter datasheet ESP32-S3 pada 240 MHz, konsumsi daya selama inferensi diperkirakan sebesar 549 mW (chip). Tabel 6 menyajikan rincian konsumsi daya per skenario. Daya ini setara dengan drone mikro yang beroperasi selama 1-2 jam dengan baterai 2S 1.000 mAh.

Table 6: Estimasi Konsumsi Daya ESP32-S3 per Skenario

Skenario	Arus (mA)	Chip (mW)	USB (mW)
Deep sleep	25.0	82.5	97.1
Idle (kamera)	75.0	247.5	291.2
Idle (tanpa kam)	25.0	82.5	97.1
Kamera + WiFi	356.5	1176.5	1384.1
Inferensi saja	166.5	549.4	646.4
Inferensi + Kam	216.5	714.4	840.5
Pipeline penuh	396.5	1308.4	1539.4

Daya USB diestimasi pada efisiensi LDO 85%

4.5 Perbandingan Lintas Platform dan Pembahasan

Tabel 7 menyajikan perbandingan kinerja GPU (RTX 3060) dengan tiga konfigurasi ESP32-S3. GPU memberikan 421,6 FPS dengan latensi 2,37 ms, sedangkan ESP32-S3 optimal memberikan 0,139 FPS. Perbedaan 4 orde magnitudo ini konsisten dengan perbedaan daya komputasi teoretis (GPU: 12 TFLOPS vs ESP32-S3: 1 GOPS untuk INT8).

Analisis menunjukkan bahwa setelah optimasi LOGISTIC, bottleneck utama kini adalah Conv2D (92% latensi). ESP-NN Conv2D sudah menggunakan SIMD Xtensa LX7 secara penuh; percepatan lebih lanjut memerlukan perubahan arsitektur model seperti backbone lebih dangkal, pruning terstruktur, atau resolusi input lebih rendah.

4.6 Limitasi

Penelitian ini memiliki beberapa keterbatasan. Pertama, akurasi 4,41% mAP@0.5 membatasi penggunaan otonom. Kedua, latensi 7,2 detik per frame belum memenuhi syarat real-time. Ketiga, estimasi daya bersifat teoretis berdasarkan datasheet. Keempat, belum ada integrasi kamera atau pipeline end-to-end pada hardware fisik.

5 Kesimpulan

Penelitian ini mendemonstrasikan bahwa model YOLOv8n yang dikuantisasi INT8 dapat dijalankan pada mikrokontroler ESP32-S3 dengan latensi 7,2 detik per inferensi dan konsumsi daya 549 mW, melalui optimasi progressive berbasis profil operasi yang menghasilkan percepatan total $30,4\times$ dari baseline referensi. Tensor

arena terukur 510 KB mengonfirmasi kelayakan memori. Temuan utama bahwa operasi LOGISTIC menyumbang 62,3% latensi sebelum optimasi menunjukkan pentingnya profiling operasi dalam pengembangan sistem TinyML.

Penelitian selanjutnya dapat berfokus pada: (i) arsitektur model yang lebih efisien untuk mengurangi beban Conv2D, (ii) evaluasi trade-off resolusi-akurasi pada 160×160 dan 128×128 , (iii) integrasi kamera OV2640 untuk pipeline end-to-end, dan (iv) pengukuran daya empiris menggunakan sensor INA219.

References

Table 7: Cross-Platform Performance Comparison (INT8 Quantized Model)

Platform	Resolution	FPS	Latency (ms)	Power (W)
PC (GPU)	192x192	421.6	2.37	—
ESP32-S3 [†]	192×192	0.0046	219287	0.55 [‡]
ESP32-S3 [§]	192×192	0.055	18020	0.55 [‡]
ESP32-S3 [¶]	192×192	0.139	7216	0.55 [‡]

MHz.

[§] With ESP-NN optimized Conv2D (12.2× speedup).

[¶] With ESP-NN Conv2D + LUT-based LOGISTIC (30.4× total, this work).

[‡] Model-based estimate; no empirical power measurement.

[†] Reference TFLite Micro kernels on ESP32-S3 @ 240